



# Il sistema di prenotazione Moove

Analisi logica del processo che decide se consentire o negare il noleggio di un MooveBike, di un MooveScooter o di un MooveKick.

Diagramma di flusso

Tavole di verità

Pseudocodice

Casi di test

**Raffaele Feola**

Progetto di Fondamenti di sviluppo · Settembre 2026



**MooveBike**

bicicletta elettrica



**MooveScooter**

scooter elettrico



**MooveKick**

monopattino elettrico

# Che cosa contiene questo elaborato

01

## Contesto e obiettivo

Moove, il servizio e il problema da modellare

02

## Le tre categorie di mezzo

MooveBike · MooveScooter · MooveKick e i loro parametri

03

## Le condizioni del sistema

Formalizzazione booleana della decisione

04

## Diagramma di flusso

Parte 1: accesso e scelta del mezzo · Parte 2: verifiche ed esito

05

## Tavole di verità

Tre condizioni, versione estesa e confronto fra i tre mezzi

06

## Pseudocodice

Configurazione per tipo, funzioni di verifica, procedura principale

07

## Casi di test

Input, esito atteso e riga della tavola di verità

08

## Riflessione sulla qualità del codice

Chiarezza, organizzazione, riusabilità e limiti noti

# Moove: micromobilità condivisa

## Vision

Rendere la mobilità urbana più sostenibile, accessibile e condivisa, migliorando la vita quotidiana delle persone.

## Mission

Offrire soluzioni di micromobilità flessibili, convenienti ed ecologiche, per ridurre il traffico e l'impatto ambientale degli spostamenti in città.

## Chi siamo

Presenza in 20 città europee con una flotta interamente elettrica di bici, scooter e monopattini a parcheggio libero.

## IL PERCORSO DELL'UTENTE NELL'APP

### 1 • Geolocalizzazione

L'app individua i mezzi liberi più vicini all'utente.

### 2 • Scelta del mezzo

L'utente sceglie la categoria e il veicolo specifico.

### 3 • Verifiche

Il sistema controlla account, disponibilità e requisiti.

### 4 • Pagamento

Tariffa oraria, giornaliera o abbonamento mensile.

### 5 • Sblocco

Codice di sblocco e avvio del noleggio.

**Il punto critico è il passo 3.** È lì che il sistema prende una decisione binaria — **consentire** o **negare** la prenotazione — combinando più condizioni indipendenti fra loro. È questa logica che l'elaborato modella.

# Una sola domanda, quattro condizioni

## «Questo utente può prenotare questo mezzo, adesso?»

La risposta è un valore booleano: **VERO** (prenotazione consentita) oppure **FALSO** (prenotazione negata). Non esistono vie di mezzo: il sistema è **conservativo** e nega ogni volta che anche una sola condizione non è soddisfatta.

Prenotazione =

**A** AND **B** AND **R** AND **C**

### CONDIZIONE A

#### Account attivo

Utente registrato, identità verificata, non sospeso e senza insoluti.

### CONDIZIONE B

#### Mezzo del tipo scelto disponibile

Nella flotta MooveBike / MooveScooter / MooveKick: stato «libero» e batteria sopra la soglia del tipo.

### CONDIZIONE R · specifica del tipo

#### Requisiti della categoria

Età minima e, per MooveScooter, patente valida: cambiano da un tipo di mezzo all'altro.

### CONDIZIONE C

#### Pagamento completato

Metodo di pagamento valido e transazione autorizzata dal gestore.

# Tre categorie, tre configurazioni

Ogni tipo di mezzo porta con sé i propri parametri: la logica di decisione resta una sola.



## MooveBike

Bicicletta elettrica a pedalata assistita

|                 |               |
|-----------------|---------------|
| Codice mezzo    | MB-000        |
| Età minima      | 14 anni       |
| Patente         | Non richiesta |
| Batteria minima | 20 %          |
| Tariffa         | 0,15 €/min    |
| Casco           | Consigliato   |



## MooveScooter

Scooter elettrico 50 cc equivalenti

|                 |                    |
|-----------------|--------------------|
| Codice mezzo    | MS-000             |
| Età minima      | 18 anni            |
| Patente         | AM o B — richiesta |
| Batteria minima | 35 %               |
| Tariffa         | 0,29 €/min         |
| Casco           | Obbligatorio       |



## MooveKick

Monopattino elettrico

|                 |                        |
|-----------------|------------------------|
| Codice mezzo    | MK-000                 |
| Età minima      | 16 anni                |
| Patente         | Non richiesta          |
| Batteria minima | 25 %                   |
| Tariffa         | 0,19 €/min             |
| Casco           | Obbligatorio < 18 anni |

**Perché questa suddivisione conta.** I tre tipi non cambiano *quali* controlli il sistema esegue, ma solo i **valori di soglia** con cui li esegue. Trattandoli come una tabella di configurazione, aggiungere domani un **MooveCargo** significa aggiungere una riga — non riscrivere la logica.

# Dalle parole alle variabili booleane

| VAR | SIGNIFICATO                       | È VERA QUANDO...                                                                                    |
|-----|-----------------------------------|-----------------------------------------------------------------------------------------------------|
| A   | Account attivo                    | <code>utente.stato = "attivo"</code>                                                                |
| B   | Mezzo del tipo scelto disponibile | <code>mezzo.stato = "libero" AND<br/>mezzo.batteria ≥ soglia_tipo</code>                            |
| R   | Requisiti della categoria         | <code>utente.età ≥ età_min_tipo AND<br/>(NOT patente_richiesta OR<br/>utente.patente_valida)</code> |
| C   | Pagamento completato              | <code>esito_pagamento = "riuscito"</code>                                                           |

**Ordine dei controlli.**  $A \rightarrow B \rightarrow R \rightarrow C$  non è casuale: si parte dal controllo più economico (una lettura sul database) e si arriva per ultimo al **pagamento**, l'unica operazione irreversibile e con un costo reale. Addebitare per poi scoprire che il mezzo è occupato significherebbe dover rimborsare.

## Valutazione a cortocircuito

Appena una condizione risulta falsa il sistema esce subito con un messaggio specifico, senza valutare le successive.

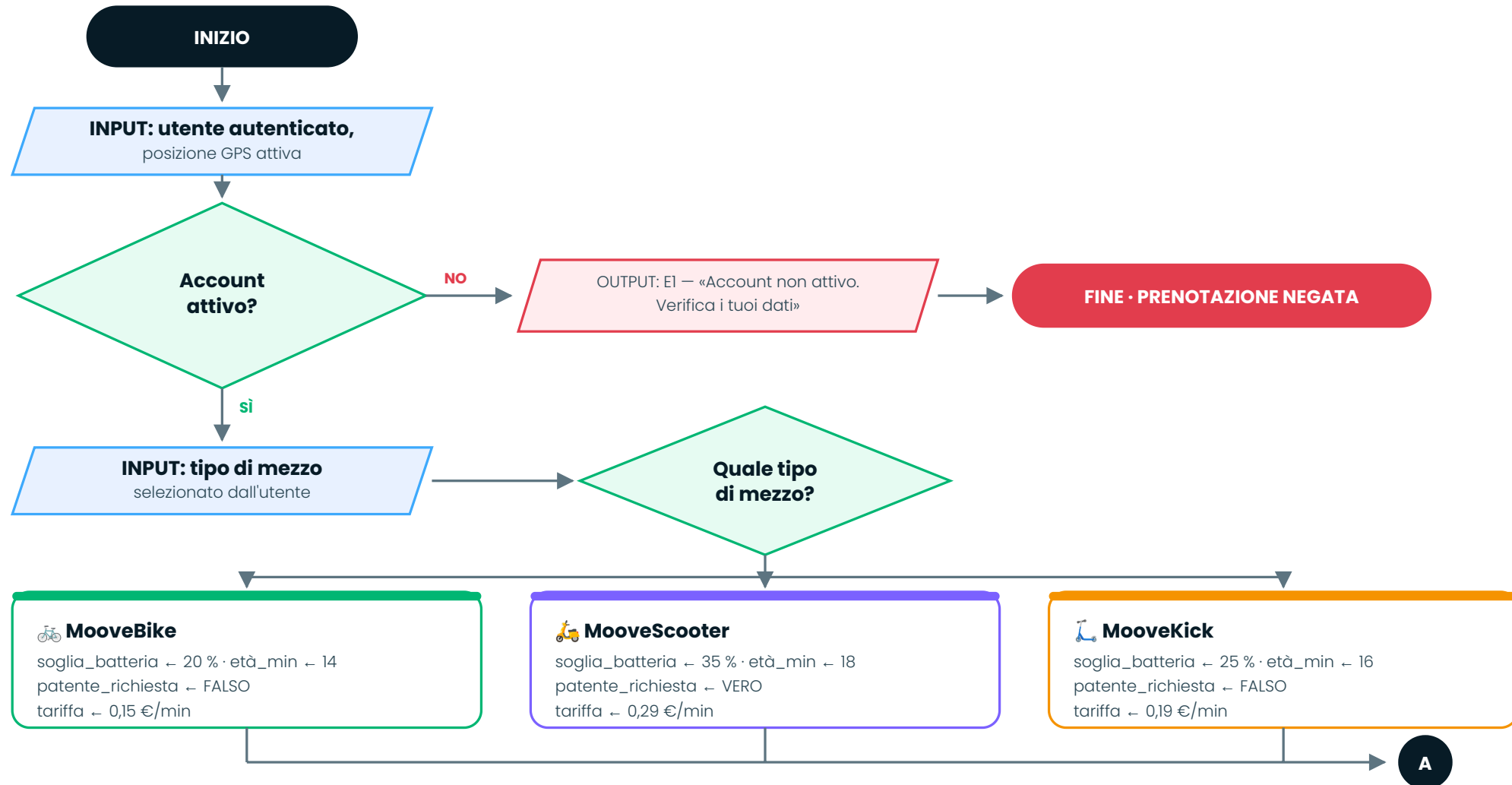
- E1** Account non attivo
- E2** Nessun mezzo del tipo scelto disponibile
- E3** Requisiti della categoria non soddisfatti
- E4** Pagamento non riuscito

**OK** Prenotazione confermata + codice di sblocco

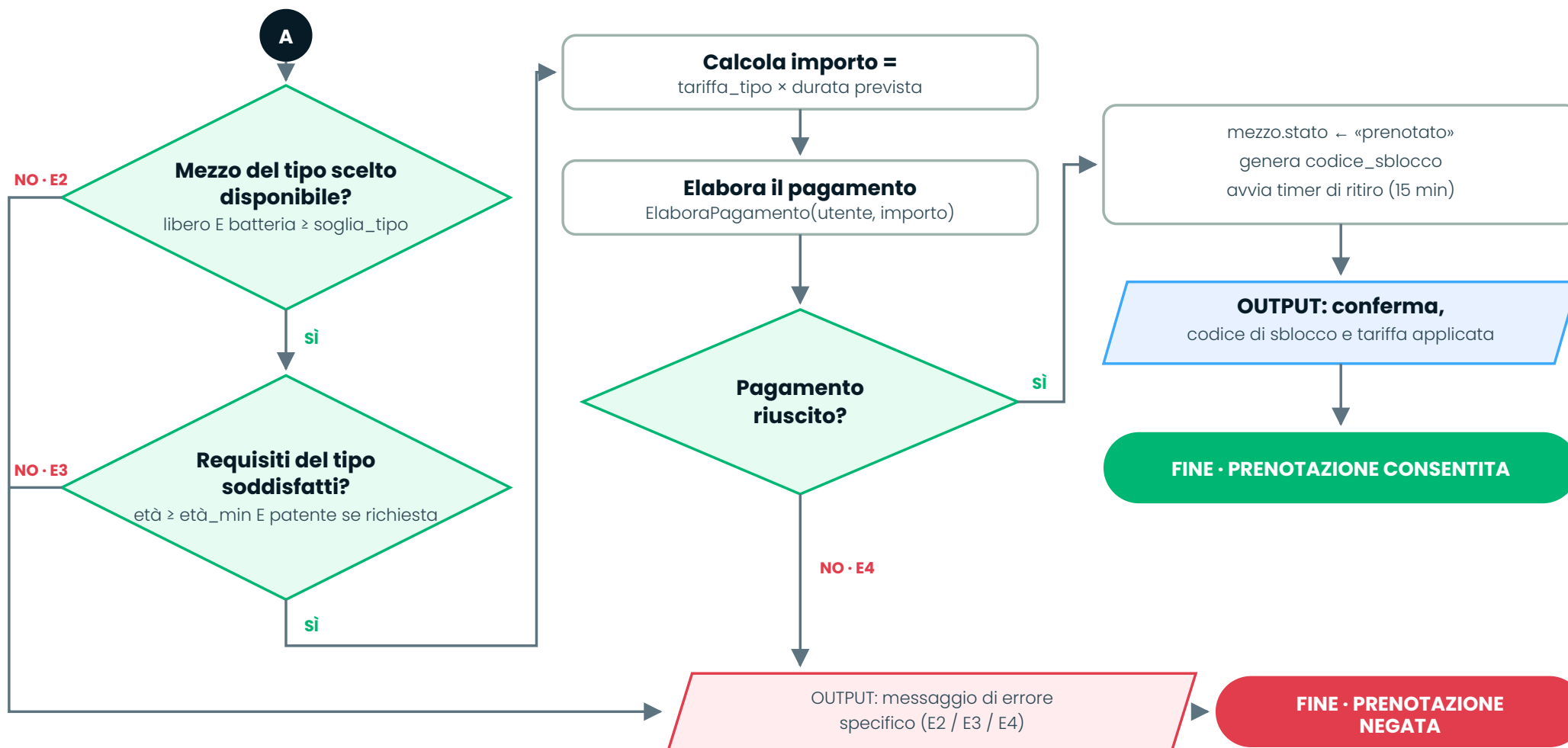
Quattro messaggi distinti invece di un generico «prenotazione negata»: l'utente capisce **che cosa fare** per risolvere.

**Nota.** La tavola di verità descrive la **logica** (tutte le combinazioni possibili); il diagramma di flusso descrive l'**esecuzione**, dove il cortocircuito impedisce che alcune combinazioni vengano mai valutate.

# Accesso e selezione della categoria



# Verifiche, pagamento ed esito finale



# Le tre condizioni principali

A = account attivo · B = mezzo del tipo scelto disponibile · C = pagamento riuscito

| # | A | B | C | ESITO DELLA PRENOTAZIONE                         |
|---|---|---|---|--------------------------------------------------|
| 1 | V | V | V | ✓ <b>CONSENTITA</b> — codice di sblocco generato |
| 2 | V | V | F | ✗ Negata — E4 pagamento non riuscito             |
| 3 | V | F | V | ✗ Negata — E2 mezzo non disponibile              |
| 4 | V | F | F | ✗ Negata — E2 (il pagamento non viene tentato)   |
| 5 | F | V | V | ✗ Negata — E1 account non attivo                 |
| 6 | F | V | F | ✗ Negata — E1                                    |
| 7 | F | F | V | ✗ Negata — E1                                    |
| 8 | F | F | F | ✗ Negata — E1                                    |

## Che cosa dice la tavola

- ☐ **1 riga su 8** produce un esito positivo: è il comportamento tipico dell'operatore **AND**.
- ☐ Il sistema è **conservativo**: in caso di dubbio nega. Per un servizio che sblocca veicoli su strada è la scelta corretta.
- ☐ Le righe 4, 6, 7 e 8 mostrano che il valore di C è **irrilevante** quando A o B sono falsi: nell'esecuzione reale il pagamento non viene nemmeno tentato.
- ☐ Il numero di righe è **2<sup>n</sup>**: con 3 condizioni sono 8, con 4 diventano 16.

**Espressione booleana equivalente:** Prenotazione =  $A \cdot B \cdot C$  — la riga 1 è l'unico *mintermine* che vale 1. Negando l'espressione si ottiene la condizione di rifiuto:  $\text{NOT}(A \cdot B \cdot C) = \text{NOT } A + \text{NOT } B + \text{NOT } C$ , cioè «basta che una sola condizione fallisca».

# Versione estesa con i requisiti di categoria

Si aggiunge R = requisiti del tipo di mezzo (età minima, patente) →  $2^4 = 16$  combinazioni

| # | A | B | R | C | ESITO        |
|---|---|---|---|---|--------------|
| 1 | V | V | V | V | ✓ CONSENTITA |
| 2 | V | V | V | F | ✗ E4         |
| 3 | V | V | F | V | ✗ E3         |
| 4 | V | V | F | F | ✗ E3         |
| 5 | V | F | V | V | ✗ E2         |
| 6 | V | F | V | F | ✗ E2         |
| 7 | V | F | F | V | ✗ E2         |
| 8 | V | F | F | F | ✗ E2         |

| #  | A | B | R | C | ESITO |
|----|---|---|---|---|-------|
| 9  | F | V | V | V | ✗ E1  |
| 10 | F | V | V | F | ✗ E1  |
| 11 | F | V | F | V | ✗ E1  |
| 12 | F | V | F | F | ✗ E1  |
| 13 | F | F | V | V | ✗ E1  |
| 14 | F | F | V | F | ✗ E1  |
| 15 | F | F | F | V | ✗ E1  |
| 16 | F | F | F | F | ✗ E1  |

**Lettura del risultato.** Su 16 combinazioni una sola porta alla conferma: **Prenotazione = A · B · R · C**. La colonna «Esito» riporta anche il messaggio restituito, che coincide sempre con la **prima** condizione falsa incontrata nell'ordine A → B → R → C: è così che la tavola di verità e il diagramma di flusso restano coerenti.

# Stesso utente, tre mezzi, tre esiti diversi

È la condizione R a rendere la decisione dipendente dalla categoria scelta

## Profilo utente di riferimento

Giulia, 17 anni · account attivo e verificato · senza patente · carta di credito valida · si trova in Piazza Duomo, dove sono liberi un MooveBike (batteria 62 %), un MooveScooter (batteria 80 %) e un MooveKick (batteria 41 %).

| MEZZO SCELTO                                                                                                 | A · ACCOUNT | B · DISPONIBILITÀ | R · REQUISITI DEL TIPO               | C · PAGAMENTO | ESITO                                                             |
|--------------------------------------------------------------------------------------------------------------|-------------|-------------------|--------------------------------------|---------------|-------------------------------------------------------------------|
|  <b>MooveBike</b> MB-118    | V           | V<br>62 % ≥ 20 %  | V<br>17 ≥ 14 · patente non richiesta | V             | ✓ <b>CONSENTITA</b>                                               |
|  <b>MooveScooter</b> MS-042 | V           | V<br>80 % ≥ 35 %  | F<br>17 < 18 · patente mancante      | —             | ✗ <b>Negata</b> — E3 «Requisiti non soddisfatti per MooveScooter» |
|  <b>MooveKick</b> MK-207    | V           | V<br>41 % ≥ 25 %  | V<br>17 ≥ 16 · patente non richiesta | V             | ✓ <b>CONSENTITA</b>                                               |

**Conclusione.** Le condizioni A, B, R e C restano **sempre le stesse**: cambiano solo i valori di soglia caricati in base al tipo di mezzo. Un unico algoritmo governa tre categorie di veicoli — e ne governerebbe dieci con la stessa struttura.

# Configurazione per tipo e funzioni di verifica

```
// — 1. Parametri specifici di ogni categoria —
FUNZIONE CaricaParametri(tipo_mezzo) RESTITUISCE configurazione
    SELEZIONA CASO tipo_mezzo

    CASO "MooveBike"
        soglia_batteria ← 20
        eta_minima ← 14
        patente_richiesta ← FALSO
        tariffa_al_minuto ← 0.15

    CASO "MooveScooter"
        soglia_batteria ← 35
        eta_minima ← 18
        patente_richiesta ← VERO
        tariffa_al_minuto ← 0.29

    CASO "MooveKick"
        soglia_batteria ← 25
        eta_minima ← 16
        patente_richiesta ← FALSO
        tariffa_al_minuto ← 0.19

    ALTRIMENTI
        SEGNALE ERRORE "Tipo di mezzo non riconosciuto"
    FINE SELEZIONA

    RESTITUISCI (soglia_batteria, eta_minima,
                patente_richiesta, tariffa_al_minuto)
    FINE FUNZIONE
```

```
// — 2. Condizione A —
FUNZIONE AccountAttivo(utente) RESTITUISCE booleano
    RESTITUISCI utente.stato = "attivo"
        E utente.identita_verificata
        E NON utente.ha_insoluti
    FINE FUNZIONE

// — 3. Condizione B —
FUNZIONE MezzoDisponibile(mezzo, cfg) RESTITUISCE booleano
    RESTITUISCI mezzo.stato = "libero"
        E mezzo.batteria >= cfg.soglia_batteria
    FINE FUNZIONE

// — 4. Condizione R —
FUNZIONE RequisitiSoddisfatti(utente, cfg) RESTITUISCE booleano
    SE utente.eta < cfg.eta_minima ALLORA
        RESTITUISCI FALSO
    FINE SE
    SE cfg.patente_richiesta E NON utente.patente_valida ALLORA
        RESTITUISCI FALSO
    FINE SE
    RESTITUISCI VERO
    FINE FUNZIONE

// — 5. Condizione C —
FUNZIONE ElaboraPagamento(utente, importo) RESTITUISCE esito
    // delega al gestore dei pagamenti
    RESTITUISCI Gateway.addebita(utente.metodo, importo)
    FINE FUNZIONE
```

# La procedura principale

```
PROCEDURA PrenotaMezzoMoove(utente, tipo_mezzo,
                             mezzo, durata_prevista)
```

```
// --- Condizione A : account ---
```

```
SE NON AccountAttivo(utente) ALLORA
    MOSTRA "E1 · Account non attivo."
    RESTITUISCI PRENOTAZIONE_NEGATA
FINE SE
```

```
// --- Ramificazione per categoria di mezzo ---
cfg ← CaricaParametri(tipo_mezzo)
```

```
// --- Condizione B : disponibilità nel tipo ---
```

```
SE NON MezzoDisponibile(mezzo, cfg) ALLORA
    MOSTRA "E2 · Nessun " + tipo_mezzo
        + " disponibile. Scegliere un altro."
    RESTITUISCI PRENOTAZIONE_NEGATA
FINE SE
```

```
// --- Condizione R : requisiti della categoria ---
```

```
SE NON RequisitiSoddisfatti(utente, cfg) ALLORA
    MOSTRA "E3 · Requisiti non soddisfatti per "
        + tipo_mezzo
    RESTITUISCI PRENOTAZIONE_NEGATA
FINE SE
```

```
// --- Condizione C : pagamento ---
//      ultima perché irreversibile
importo ← cfg.tariffa_al_minuto * durata_prevista
esito ← ElaboraPagamento(utente, importo)
```

```
SE esito ≠ "riuscito" ALLORA
    MOSTRA "E4 · Pagamento non riuscito."
    RESTITUISCI PRENOTAZIONE_NEGATA
FINE SE
```

```
// --- Tutte le condizioni sono VERE ---
```

```
mezzo.stato ← "prenotato"
codice_sblocco ← GeneraCodice(mezzo)
AvviaTimerRitiro(mezzo, 15) // minuti

MOSTRA "Prenotazione confermata: " + mezzo.codice
    + " · sblocco " + codice_sblocco
RESTITUISCI PRENOTAZIONE_CONFERMATA
```

```
FINE PROCEDURA
```

```
// Esempio di chiamata
```

```
PrenotaMezzoMoove(giulia, "MooveKick", MK-207, 18)
```

**Uscite anticipate.** Nessun SE annidato: ogni controllo ha una sola via d'uscita e la lettura resta lineare dall'alto verso il basso.

**La categoria è un dato, non un ramo logico.**  
CaricaParametri isola tutto ciò che distingue i tre mezzi dal resto dell'algoritmo.

**Effetti solo alla fine.** Stato del mezzo, codice di sblocco e timer si aggiornano in un unico punto, dopo l'esito positivo.

# Casi di test

Due casi obbligatori (T1 e T2) più due casi aggiuntivi per coprire ogni tipo di mezzo

| ID | INPUT                                                                                                                                                                                                              | CONDIZIONI                          | RISULTATO ATTESO                                                                                                                                                    | RIGA TDV |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| T1 |  <b>MooveBike MB-118</b><br>Marco, 24 anni, account attivo, carta valida.<br>Mezzo libero, batteria 62 %, durata prevista 20 min. | A = V   B = V<br>R = V   C = V      | <b>✓ PRENOTAZIONE CONFERMATA</b><br>Importo addebitato 3,00 € · MB-118.stato = "prenotato" · codice di sblocco generato · timer di ritiro avviato.                  | 1        |
| T2 |  <b>MooveScooter MS-042</b><br>Giulia, 17 anni, account attivo, <b>senza patente</b> .<br>Mezzo libero, batteria 80 %.            | A = V   B = V<br><b>R = F</b> C = — | <b>✗ PRENOTAZIONE NEGATA</b> — messaggio E3.<br><b>Nessun addebito</b> effettuato e MS-042.stato resta "libero": il mezzo torna disponibile per altri utenti.       | 3        |
| T3 |  <b>MooveKick MK-207</b><br>Luca, 20 anni, account attivo, carta valida.<br>Mezzo libero ma <b>batteria 12 %</b> (soglia 25 %).   | A = V <b>B = F</b><br>R = —   C = — | <b>✗ PRENOTAZIONE NEGATA</b> — messaggio E2.<br>Il controllo si ferma prima dei requisiti e del pagamento: nessuna transazione viene aperta.                        | 5        |
| T4 |  <b>MooveScooter MS-311</b><br>Elena, 29 anni, patente B, account attivo.<br>Mezzo libero, batteria 74 %, <b>carta scaduta</b> .  | A = V   B = V<br>R = V <b>C = F</b> | <b>✗ PRENOTAZIONE NEGATA</b> — messaggio E4.<br>Il mezzo <b>non</b> viene bloccato e nessun codice di sblocco viene generato: lo stato del sistema resta invariato. | 2        |

**Che cosa verificano davvero questi test.** Non solo il messaggio restituito, ma anche l'**assenza di effetti collaterali** quando la prenotazione fallisce: nessun addebito, nessun mezzo bloccato inutilmente. È la parte della logica che più facilmente si rompe quando il codice viene modificato.

# Qualità del codice: chiarezza, organizzazione, riuso

## Chiarezza

- ☐ **Nomi parlanti:** `soglia_batteria`, `RequisitiSoddisfatti` si leggono come una frase in italiano.
- ☐ **Un controllo per blocco:** ogni `SE` verifica una sola condizione e ha una sola via d'uscita.
- ☐ **Messaggi specifici** (E1-E4): l'utente sa che cosa deve correggere, non solo che ha fallito.
- ☐ **Struttura piatta:** le uscite anticipate evitano annidamenti profondi e rendono il codice leggibile dall'alto verso il basso.

## Organizzazione

- ☐ **Separazione dei ruoli:** le funzioni *decidono*, la procedura principale *coordina*, il gateway *esegue* il pagamento.
- ☐ **Configurazione separata dalla logica:** i parametri dei tre mezzi stanno in un unico punto.
- ☐ **Ordine motivato dei controlli:** dal più economico al più costoso e irreversibile.
- ☐ **Effetti finali raggruppati** alla fine, così lo stato del sistema cambia in un solo punto identificabile.

## Riusabilità

- ☐ **Nuovo mezzo = nuova riga:** un **MooveCargo** o un **MooveMoped** si aggiungono a `CaricaParametri` senza toccare l'algoritmo.
- ☐ **Funzioni indipendenti:** `AccountAttivo` serve anche al login, al noleggio e all'assistenza clienti.
- ☐ **Schema generalizzabile:** «verifica i requisiti → esegui la transazione → aggiorna lo stato» vale per il car sharing, una camera d'albergo o il noleggio di attrezzature.
- ☐ **Indipendenza dal linguaggio:** lo pseudocodice si traduce quasi riga per riga in Python, Java o JavaScript.

# Limiti del modello e sviluppi futuri

Riconoscere che cosa il modello non copre è parte della valutazione della qualità

## LIMITI ATTUALI



### Concorrenza

Due utenti che selezionano lo stesso MooveKick nello stesso istante potrebbero superare entrambi il controllo B. Serve un blocco temporaneo del record.



### Nessun rollback

Se il pagamento riesce ma l'aggiornamento dello stato del mezzo fallisce, l'utente viene addebitato senza ricevere il veicolo.



### Scadenza della prenotazione

Il timer di 15 minuti è avviato ma il modello non descrive che cosa accade quando scade.



### Abbonamenti

Con un abbonamento mensile attivo il pagamento non andrebbe rielaborato a ogni corsa: la condizione C va estesa.

## ESTENSIONI POSSIBILI



### Prenotazione transazionale

Racchiudere pagamento e aggiornamento dello stato in un'unica operazione atomica: o riescono entrambi, o si annulla tutto.



### Suggerimento alternativo

Quando la condizione R fallisce, proporre automaticamente le categorie compatibili con l'utente (nel caso di Giulia: MooveBike e MooveKick).



### Condizione geografica

Aggiungere il controllo sulla distanza massima e sulle aree a traffico limitato, diverse per ciascuna categoria.



### Registro delle decisioni

Tracciare quale condizione ha causato ogni rifiuto per analizzare dove il servizio perde clienti.

IN SINTESI

# Una decisione binaria, quattro condizioni, tre categorie di mezzo.

1 / 16

combinazioni che portano alla conferma: il sistema nega ogni volta che ha un dubbio.

4

messaggi distinti, così l'utente sa sempre che cosa deve correggere.

1

algoritmo per MooveBike, MooveScooter e MooveKick: la categoria è un parametro, non un caso a parte.



**moove**

Raffaele Feola · Progetto di Fondamenti di sviluppo · Settembre 2026